React useEffect Hook Tutorial

Introduction

The `useEffect` hook is one of the most important hooks in React. It allows you to perform side effects in functional components, such as data fetching, subscriptions, or manually changing the DOM. It serves the same purpose as `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount` combined in class components.

Basic Usage

Here's how you use the `useEffect` hook:

```
import React, { useState, useEffect } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(seconds => seconds + 1);
    }, 1000);

    // Cleanup function
    return () => clearInterval(interval);
  }, []); // Empty dependency array means this effect runs once

  return (
    <div>
      <p>Timer: {seconds} seconds</p>
    </div>
  );
}
```

Explanation: - `useEffect` takes a function as its first argument, which contains the side effect. - The second argument is the dependency array `[]`. - The cleanup function (returned function) runs when the component unmounts or before the effect runs again.

---

useEffect with Dependencies

You can control when the effect runs by providing dependencies:

```
import React, { useState, useEffect } from 'react';

import React, { useState, useEffect } from 'react';


function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    setLoading(true);

    // Simulate API call
    setTimeout(() => {
      setUser({ id: userId, name: `User ${userId}` });
      setLoading(false);
    }, 1000);
  }, [userId]); // Effect runs when userId changes

  if (loading) return <p>Loading...</p>;

  return (
    <div>
      <h2>{user.name}</h2>
      <p>User ID: {user.id}</p>
    </div>
  );
}

export function App(props) {
  return (
    <div className='App'>
    <UserProfile userId='mrgh' />
    </div>
  );
}
```

Explanation: - The effect runs whenever `userId` changes. - If `userId` stays the same, the eff ect won't run again. - This prevents unnecessary API calls.

Different Types of useEffect

1. Effect that runs on every render

```
useEffect(() => {
  console.log('Runs after every render');
});
```

2. Effect that runs only once (on mount)

```
useEffect(() => {
  console.log('Runs only once after initial render');
}, []);
```

3. Effect with dependencies

```
useEffect(() => {
  console.log('Runs when count changes');
}, [count]);
```

4. Effect with cleanup

```
useEffect(() => {
  const subscription = subscribeToSomething();

  return () => {
    subscription.unsubscribe();
  };
}, []);
```

Real-World Example: Data Fetching

```
import React, { useState, useEffect } from 'react';

function PostsList() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchPosts = async () => {
      try {
        setLoading(true);
        const response = await
fetch('https://jsonplaceholder.typicode.com/posts');
        const data = await response.json();
        setPosts(data.slice(0, 5)); // Get first 5 posts
      } catch (err) {
        setError('Failed to fetch posts');
      } finally {
        setLoading(false);
      }
    };
    fetchPosts();
  }, []);
  if (loading) return <p>Loading posts...</p>;
  if (error) return <p>Error: {error}</p>;
  return (
    <div>
      <h2>Posts</h2>
      {posts.map(post => (
        <div key={post.id} style={{ border: '1px solid #ccc',
margin: '10px', padding: '10px' }}>
          <h3>{post.title}</h3>
          <p>{post.body}</p>
        </div>
      ))}
    </div>
  );
}
export function App(props) {
  return (
    <div className='App'>
    <PostsList />
    </div>
  );
}
```

Explanation: - This component fetches posts when it mounts. - It handles loading and erro r states. - The empty dependency array [ ] ensures the fetch only happens once.

Exercise 1

Task: Create a React component called `RandomQuoteGenerator` that:

- Uses `useState` to store a quote object with `text` and `author` properties
- Uses `useState` to handle loading and error states
- Uses `useEffect` to fetch a random quote when the component mounts
- Displays the quote and author
- Includes a "Get New Quote" button that fetches a new random quote

API to use:

- Endpoint: `https://api.quotable.io/random`
- Returns: `{ content: "quote text", author: "author name" }`

Expected Behaviour:

- Shows loading state while fetching
- Displays quote and author when loaded
- Shows error message if fetch fails
- Button refreshes the quote

Starter Code:

```
import React, { useState, useEffect } from 'react';

function RandomQuoteGenerator() {
  // Your code here

  return (
    <div>
      {/* Your code here */}
    </div>
  );
}

export default RandomQuoteGenerator;
```

Hints:

- Use `fetch()` to make API calls
- Handle async operations with try-catch blocks
- Consider creating a separate function for fetching quotes that can be reused
- Don't forget to handle loading states during button clicks

Exercise 2

Task: Create a React component called `RandomQuoteGenerator` that: - Uses `useState` to store a quote object with `text` and `author` properties - Uses `useState` to handle loading and error states - Uses `useEffect` to fetch a random quote when the component mounts - Displays the quote and author - Includes a "Get New Quote" button that fetches a new random quote

API to use: - Endpoint: `https://api.quotable.io/random` - Returns: `{ content: "quote text", author: "author name" }`

Expected Behavior: - Shows loading state while fetching - Displays quote and author when loaded - Shows error message if fetch fails - Button refreshes the quote

Starter Code:

```
import React, { useState, useEffect } from 'react';

function RandomQuoteGenerator() {
  // Your code here

  return (
    <div>
      {/* Your code here */}
    </div>
  );
}
```

```
export default RandomQuoteGenerator;
```

Hints: - Use `fetch()` to make API calls - Handle async operations with try-catch blocks - Consider creating a separate function for fetching quotes that can be reused - Don't forget to handle loading states during button clicks

Exercise 3

Task: Create a React component called `WindowSizeTracker` that: - Uses `useState` to store the window width and height. - Uses `useEffect` to add an event listener for the `resize` event. - Displays the current window dimensions. - Properly cleans up the event listener when the component unmounts.

Expected Behavior: - Shows current window size (e.g., "Window size: 1200 x 800") - Updates in real-time when the window is resized

Starter Code:

```
import React, { useState, useEffect } from 'react';

function WindowSizeTracker() {
  // Your code here

  return (
    <div>
      {/* Your code here */}
    </div>
  );
}

export default WindowSizeTracker;
```

Hints: - Use `window.innerWidth` and `window.innerHeight` to get dimensions - Add event listener with `window.addEventListener('resize', handler)` - Remove event listener in cleanup function with `window.removeEventListener('resize', handler)`

Common Pitfalls and Best Practices

❌ Don't do this (infinite loop):

```
function BadExample() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setCount(count + 1); // This will cause infinite re-renders
  });

  return <p>Count: {count}</p>;
}
```

✅ Do this instead:

```
function GoodExample() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setCount(c => c + 1); // Use functional update
  }, []); // Add dependency array

  return <p>Count: {count}</p>;
}
```

Best Practices:

1. Always include dependencies in the dependency array
2. Use the ESLint plugin `react-hooks/exhaustive-deps` to catch missing dependencies
3. Clean up subscriptions, timers, and event listeners
4. Use multiple `useEffect` hooks to separate concerns

Summary

- `useEffect` allows you to perform side effects in functional components
- The dependency array controls when the effect runs:
  - No array: runs after every render
  - Empty array `[]`: runs only once (on mount)
  - With dependencies `[dep1, dep2]`: runs when dependencies change
- Always clean up subscriptions, timers, and event listeners
- Use separate `useEffect` hooks for different concerns

For more information, see the React documentation on useEffect.